

Discussion 5

Self-Attention, Transformers, and kNNs

This Week in AI

- Personal agents (Grok Bot, Meta Muse, OpenAI Dots)
- More new models (Sonnet 5.5, Sol 6.1)

Self-attention: Intuition

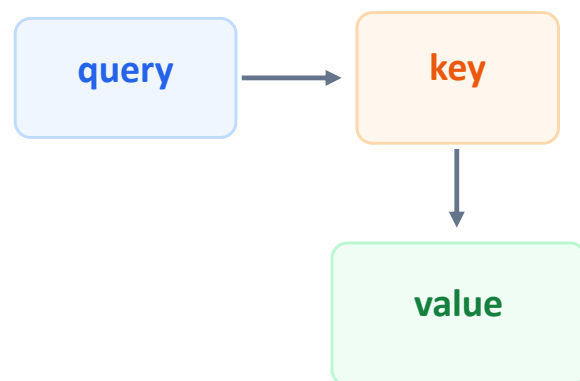
- Consider you have a dictionary
- The high-level mechanisms you do in a dictionary search are as follows:
 - Match keys – the key with the “highest similarity” is a good fit
 - Return value
- We want a similar mechanism that uses words and learns something
 - Input: words of a sentence
 - Output: a weighted combination of the most relevant information from other tokens

Self-attention is a soft dictionary lookup

Start from the mechanics of a dictionary, then soften the lookup.

Ordinary dictionary

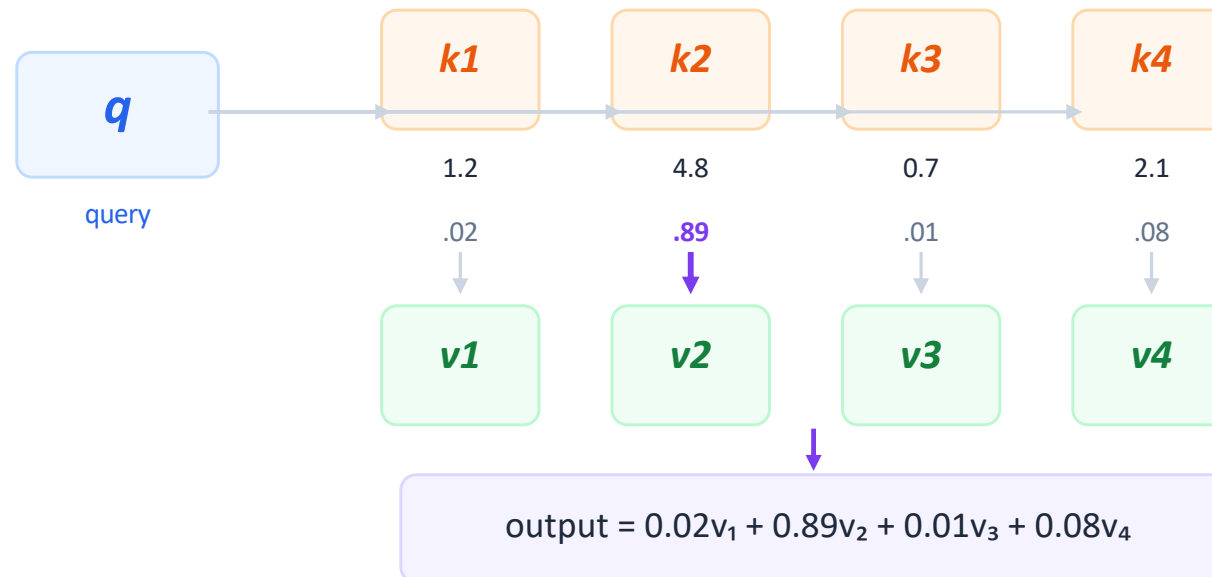
Exact key $\rightarrow$ one value



Match exactly, then return the stored value.

Self-attention

Similarity to many keys $\rightarrow$ weighted mixture of values

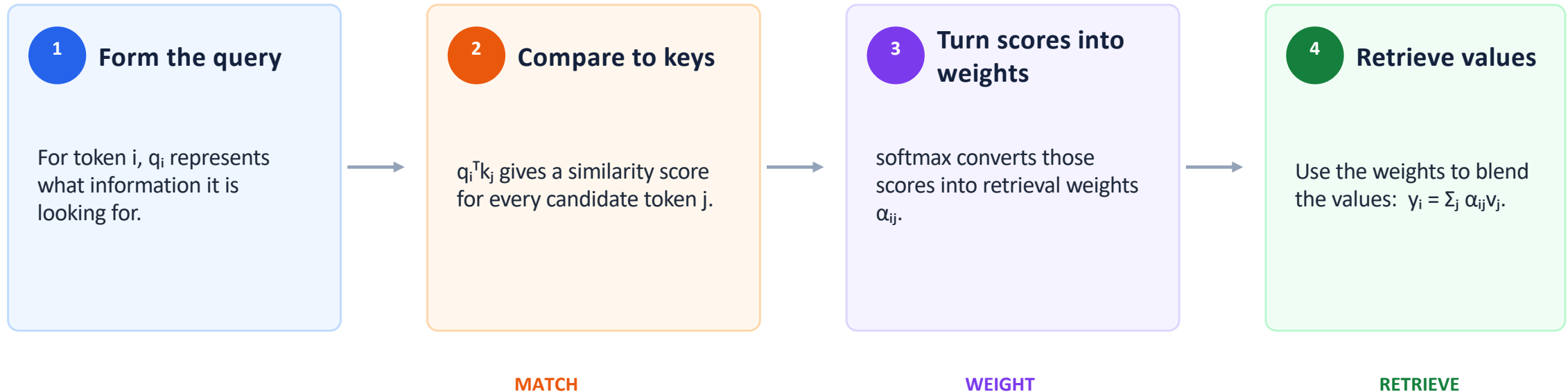


Key idea: attention does not pick one exact match — it scores all matches and blends their values.

Now read the self-attention equation as a lookup

Each part of the equation corresponds directly to one step in the dictionary search.

$$Y = \text{softmax}(QK^T / \sqrt{d_k}) V$$



QK^T answers “who is relevant?” → softmax answers “how much?” → V answers “what information do I take?”

Worked example: follow one query through attention

Assume the learned projections already produced these toy 2-D q, k, v vectors.

Sentence

The

cat

sat

← compute the output for “sat”

1 Start from q, k, v

token	q_i	k_i	v_i
The	[0, 1]	[0, 1]	[0, 0.5]
cat	[0.7, 0.7]	[1, 0]	[1, 0]
sat	[1, 0]	[0.2, 0.8]	[0, 1]

Goal: $y_{\text{sat}} = \text{softmax}(q_{\text{sat}}K^T / \sqrt{2})V$

2 Match → weight → retrieve

MATCH

$$q_{\text{sat}} \cdot K^T / \sqrt{2} \\ = [0, 0.71, 0.14]$$

cat gets the highest score

WEIGHT

$$\text{softmax} \\ \approx [0.24, 0.49, 0.28]$$

weights sum to 1

RETRIEVE

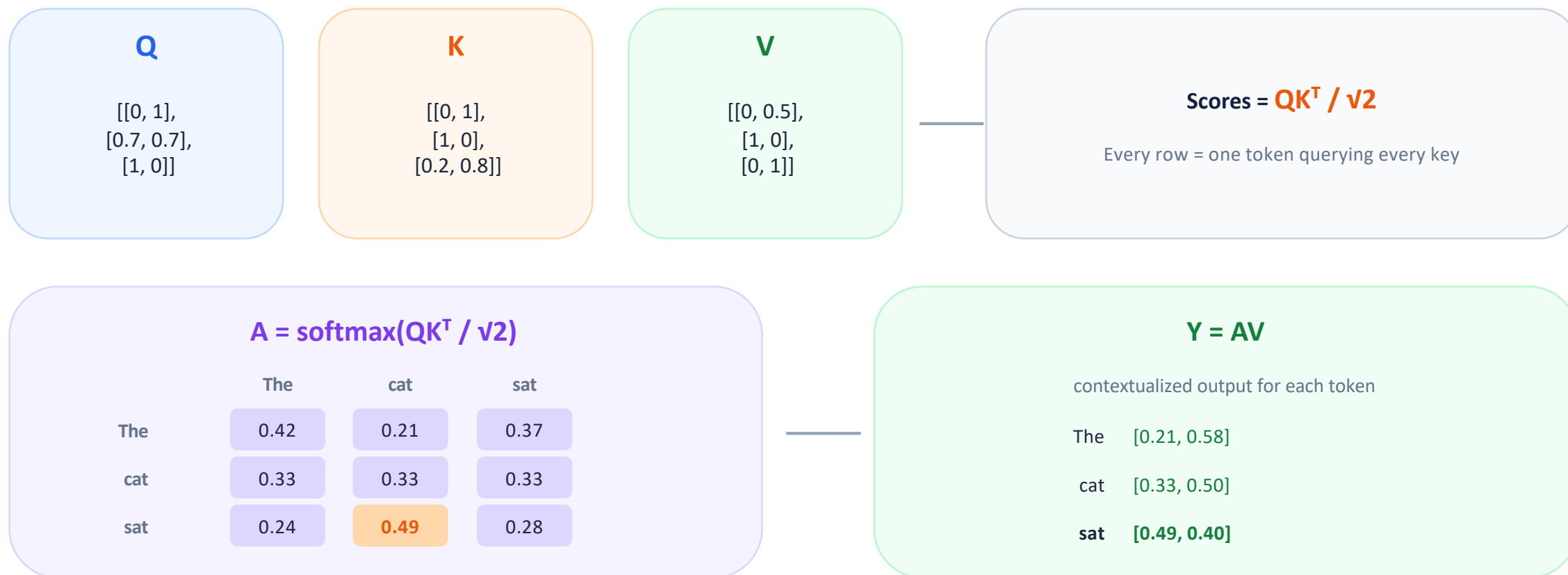
$$0.24v_1 + 0.49v_2 \\ + 0.28v_3 \\ \approx [0.49, 0.40]$$

blend the values

Interpretation: “sat” searches the three keys, gives the largest weight to “cat”, then returns a weighted mixture of the three values.

Now do the same lookup for every token at once

Stack the token queries into Q: matrix multiplication vectorizes all of the individual dictionary searches.



Key idea: one query gives one row of A; stacking all queries into Q gives the full $n \times n$ attention matrix in one operation.

Backward pass: Setup

One query at a time. The forward graph is known; the chain rule walks it in reverse.

GIVEN

$g = \partial L / \partial y$ — the gradient the rest of the network sends back into this token's output.
Cached from the forward pass: s, a, y and every q, k_j, v_j .

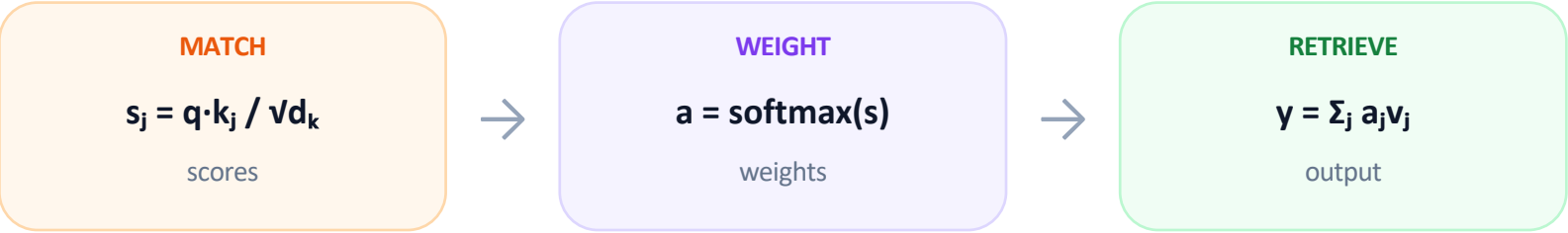
WANT

$\partial L / \partial q, \partial L / \partial k_j, \partial L / \partial v_j$ for every token j .
These are what flow into the projection weights W_Q, W_K, W_V .

SHAPES

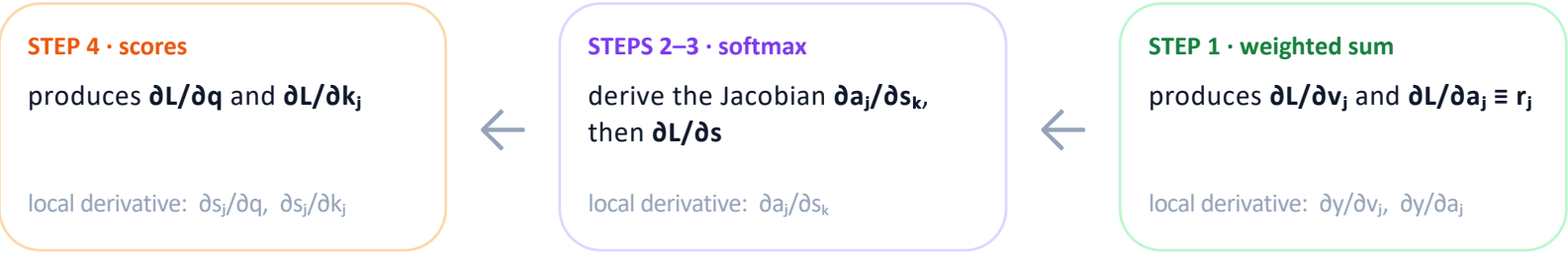
q, k_j — d_k entries each
 v_j, y, g — d_v entries each
 s, a — n entries (one per token)

FORWARD



Chain rule: $\partial L / \partial (\text{input}) = \sum \partial L / \partial (\text{output}) \cdot \partial (\text{output}) / \partial (\text{input})$
Walk the graph right-to-left, multiplying local derivatives at every box.

BACKWARD



Step 1: Backprop through the weighted sum

$y = \sum_j a_j v_j$ is linear in the weights and linear in the values, so each derivative is a single coefficient.

RETRIEVE STEP

$$y = \sum_j a_j v_j$$

upstream: $g = \partial L / \partial y$

y and g both have d_v entries

1a Gradient for a value v_j

Hold the weights fixed. v_j shows up in exactly one term of the sum:

$$y = a_1 v_1 + \dots + a_j v_j + \dots + a_n v_n \quad \Rightarrow \quad \partial y / \partial v_j = a_j$$

(a scalar times the identity). Chain rule: multiply the upstream gradient by that scalar.

$$\partial L / \partial v_j = a_j g$$

All values at once: $\partial L / \partial \mathbf{V} = \mathbf{a}^T \mathbf{g}$ ($n \times d_v$, an outer product)

1b Gradient for a weight a_j

Hold the values fixed. a_j is a scalar multiplying the vector v_j :

$$\partial y / \partial a_j = v_j$$

a_j is a scalar, so its gradient is a scalar: dot the upstream gradient with v_j .

$$\partial L / \partial a_j = g \cdot v_j \equiv r_j$$

All weights at once: $\mathbf{r} = \mathbf{V} \mathbf{g}$ (one number per token)

Hand-off to the softmax step: the only thing it needs from here is the vector $\mathbf{r} = (r_1, \dots, r_n)$.

Step 2: Derive the softmax gradient

Before we can chain through softmax we need $\partial a_j / \partial s_k$ — how weight j moves when score k moves.

SOFTMAX

$$a_j = \exp(s_j) / Z, \quad Z = \sum_m \exp(s_m)$$

2a Same index: $j = k$

Numerator and Z both contain s_j . Quotient rule, using $\partial Z / \partial s_j = \exp(s_j)$:

$$\begin{aligned} \partial a_j / \partial s_j &= [\exp(s_j) \cdot Z - \exp(s_j) \cdot \exp(s_j)] / Z^2 \\ &= \exp(s_j) / Z - [\exp(s_j) / Z]^2 \end{aligned}$$

$$= a_j (1 - a_j)$$

2b Different index: $j \neq k$

The numerator $\exp(s_j)$ has no s_k in it; only Z does, with $\partial Z / \partial s_k = \exp(s_k)$:

$$\begin{aligned} \partial a_j / \partial s_k &= [0 \cdot Z - \exp(s_j) \cdot \exp(s_k)] / Z^2 \\ &= - [\exp(s_j) / Z] \cdot [\exp(s_k) / Z] \end{aligned}$$

$$= - a_j a_k$$

2c One formula for both cases

Use the delta: $\delta_{jk} = 1$ if $j = k$, otherwise 0.

$$\partial a_j / \partial s_k = a_j (\delta_{jk} - a_k)$$

Step 3: Chain the loss through softmax

Combine $r = \partial L / \partial a$ from Step 1 with the Jacobian from Step 2.

CHAIN RULE AT SOFTMAX

$$\partial L / \partial s_k = \sum_j (\partial L / \partial a_j) \cdot (\partial a_j / \partial s_k)$$

Sum over every j : each weight a_j depends on s_k , so each one contributes.

3 Derivation

Substitute r_j for $\partial L / \partial a_j$ and the Jacobian from Step 2:

$$\partial L / \partial s_k = \sum_j r_j a_j (\delta_{jk} - a_k)$$

Split the sum into the δ part and the rest:

$$= \sum_j r_j a_j \delta_{jk} - a_k \sum_j a_j r_j$$

The δ kills every term except $j = k$:

$$= r_k a_k - a_k (a^T r)$$

$$\partial L / \partial s_k = a_k (r_k - a^T r)$$

Vector form: $\partial L / \partial s = a \odot (r - (a^T r) \cdot \mathbf{1})$ $\odot$ = elementwise

Score gradient = weight $\times$ deviation from the attention-weighted baseline — and the entries always sum to zero.

Step 4: Backprop through the scores into q and K

Each score is a scaled dot product, so its derivative with respect to one vector is just the other vector.

MATCH STEP

$$s_j = \mathbf{q} \cdot \mathbf{k}_j / \sqrt{d_k}$$

with $\partial L / \partial s_j$ known from Step 3

$\mathbf{q}$ and every $\mathbf{k}_j$ have d_k entries; the $1/\sqrt{d_k}$ just rides along

4a Gradient for the query q

$\mathbf{q}$ appears in every one of the n scores, so all n of them contribute:

$$\partial s_j / \partial \mathbf{q} = \mathbf{k}_j / \sqrt{d_k} \quad (\text{derivative of a dot product} = \text{the other vector})$$

$$\partial L / \partial \mathbf{q} = (1/\sqrt{d_k}) \sum_j (\partial L / \partial s_j) \mathbf{k}_j$$

$$\text{Matrix form: } \partial L / \partial \mathbf{q} = (\partial L / \partial \mathbf{s}) \mathbf{K} / \sqrt{d_k} \quad (1 \times n) \cdot (n \times d_k)$$

4b Gradient for a key $\mathbf{k}_j$

$\mathbf{k}_j$ appears in exactly one score, s_j ; every other score ignores it:

$$\partial s_j / \partial \mathbf{k}_j = \mathbf{q} / \sqrt{d_k}, \quad \partial s_i / \partial \mathbf{k}_j = 0 \quad \text{for } i \neq j$$

$$\partial L / \partial \mathbf{k}_j = (1/\sqrt{d_k}) (\partial L / \partial s_j) \mathbf{q}$$

$$\text{Matrix form: } \partial L / \partial \mathbf{K} = (\partial L / \partial \mathbf{s})^T \mathbf{q} / \sqrt{d_k} \quad (n \times 1) \cdot (1 \times d_k)$$

Entire Backward pass

ONE-QUERY ATTENTION

$$s_j = \mathbf{q} \cdot \mathbf{k}_j / \sqrt{d_k} \rightarrow \mathbf{a} = \text{softmax}(\mathbf{s}) \rightarrow \mathbf{y} = \sum_j \mathbf{a}_j \mathbf{v}_j$$

Given $\mathbf{g} = \partial L / \partial \mathbf{y}$

1 Through the weighted sum $\mathbf{y} = \sum_j \mathbf{a}_j \mathbf{v}_j$

Since $\mathbf{y} = \sum_j \mathbf{a}_j \mathbf{v}_j$, a small change gives

$$d\mathbf{y} = \sum_j (\mathbf{v}_j da_j + a_j d\mathbf{v}_j)$$

Dot with $\mathbf{g} = \partial L / \partial \mathbf{y}$:

$$\partial L / \partial a_j = \mathbf{g}^T \mathbf{v}_j \equiv r_j$$

$$\partial L / \partial \mathbf{v}_j = a_j \mathbf{g}$$

Matrix form: $\partial L / \partial \mathbf{V} = \mathbf{a}^T \mathbf{g}$

2 Through softmax $\mathbf{a} = \text{softmax}(\mathbf{s})$

Softmax element derivative:

$$\partial a_j / \partial s_k = a_j (\delta_{jk} - a_k)$$

Apply the chain rule:

$$\begin{aligned} \partial L / \partial s_k &= \sum_j r_j a_j (\delta_{jk} - a_k) \\ &= a_k (r_k - \sum_j a_j r_j) \end{aligned}$$

Vector form: $\partial L / \partial \mathbf{s} = \mathbf{a} \odot (\mathbf{r} - (\mathbf{a}^T \mathbf{r}) \cdot \mathbf{1})$

3 Through scores $s_j = \mathbf{q} \cdot \mathbf{k}_j / \sqrt{d_k}$

Each score is just a scaled dot product:

$$s_j = \mathbf{q} \cdot \mathbf{k}_j / \sqrt{d_k}$$

Therefore:

$$\partial L / \partial \mathbf{q} = (1 / \sqrt{d_k}) \sum_j (\partial L / \partial s_j) \mathbf{k}_j$$

$$\partial L / \partial \mathbf{k}_j = (1 / \sqrt{d_k}) (\partial L / \partial s_j) \mathbf{q}$$

Matrix: $\partial L / \partial \mathbf{Q} = (\partial L / \partial \mathbf{s}) \mathbf{K} / \sqrt{d_k}$

Practice 1: “The cat sat” — through the weighted sum

Reuse the forward pass for the query “sat”; the upstream gradient is given as $g = [1, -1]$.

FROM THE FORWARD PASS

$q_{\text{sat}} = [1, 0]$

$a = [0.239, 0.485, 0.276]$

$g = \partial L / \partial y = [1, -1]$ (given now) $d_k = 2, \sqrt{2} \approx 1.414$

Inputs

token	k_j	v_j	a_j
The	$[0, 1]$	$[0, 0.5]$	0.239
cat	$[1, 0]$	$[1, 0]$	0.485
sat	$[0.2, 0.8]$	$[0, 1]$	0.276

Only the query for “sat” is needed: it is the token whose output we backprop through.

Step 1a $\partial L / \partial v_j = a_j g$

The: $0.239 \cdot [1, -1] = [0.239, -0.239]$
cat: $0.485 \cdot [1, -1] = [0.485, -0.485]$
sat: $0.276 \cdot [1, -1] = [0.276, -0.276]$

$\partial L / \partial V = a^T g =$

$[0.239, -0.239]$
 $[0.485, -0.485]$
 $[0.276, -0.276]$

Step 1b $r_j = g \cdot v_j$

The: $1 \cdot 0 + (-1) \cdot 0.5 = -0.5$
cat: $1 \cdot 1 + (-1) \cdot 0 = 1.0$
sat: $1 \cdot 0 + (-1) \cdot 1 = -1.0$

$r = [-0.5, 1.0, -1.0]$

Step 2 needs no arithmetic: the softmax Jacobian is a formula — we apply it inside Step 3 on the next slide.

Practice 1: “The cat sat” — softmax and scores

Carry a and r forward, then push the gradient into the scores, the query, and the keys.

CARRIED OVER

$a = [0.239, 0.485, 0.276]$

$r = [-0.5, 1.0, -1.0]$

$q = [1, 0]$

K rows: $[0, 1], [1, 0], [0.2, 0.8]$

Step 3 $\partial L / \partial s_k = a_k (r_k - a^T r)$

Baseline first: $a^T r = 0.239(-0.5) + 0.485(1.0) + 0.276(-1.0)$
 $= -0.120 + 0.485 - 0.276 = \mathbf{0.090}$

The: $0.239 \times (-0.5 - 0.090) = 0.239 \times (-0.590) = -0.141$

cat: $0.485 \times (1.0 - 0.090) = 0.485 \times 0.910 = 0.442$

sat: $0.276 \times (-1.0 - 0.090) = 0.276 \times (-1.090) = -0.300$

$$\partial L / \partial s = [-0.141, 0.442, -0.300]$$

Check: $-0.141 + 0.442 - 0.300 = 0.001 \approx 0 \quad \checkmark$ (rounding only)

Step 4 into q and K

$\partial L / \partial q = (1/\sqrt{2}) [-0.141 \cdot [0, 1] + 0.442 \cdot [1, 0] - 0.300 \cdot [0.2, 0.8]]$
 $= (1/\sqrt{2}) [0.442 - 0.060, -0.141 - 0.240] = (1/\sqrt{2}) [0.382, -0.381]$

$$\partial L / \partial q \approx [0.270, -0.270]$$

$\partial L / \partial k_j = (\partial L / \partial s_j / \sqrt{2}) \cdot q$, with $q = [1, 0]$:

The: $(-0.141 / 1.414) \cdot [1, 0] = [-0.100, 0]$

cat: $(0.442 / 1.414) \cdot [1, 0] = [0.312, 0]$

sat: $(-0.300 / 1.414) \cdot [1, 0] = [-0.212, 0]$

$$\partial L / \partial K \text{ rows: } [-0.100, 0], [0.312, 0], [-0.212, 0]$$

All three gradients agree: lower s_{cat} , move q away from $k_{\text{cat}} = [1, 0]$, move k_{cat} away from q — attend to “cat” a little less.

Practice 2: “birds fly south”

Same four steps, fresh numbers. Work the output for “fly” and its gradients before flipping to the solution.

Setup

birds fly south ← query token

Query: $\mathbf{q}_{\text{fly}} = [1, 1]$ $d_k = 2$ (scale by $1/\sqrt{2}$)
Upstream gradient on the output for “fly”: $\mathbf{g} = \partial L / \partial \mathbf{y}_{\text{fly}} = [1, -1]$

token	$\mathbf{k}_j$	$\mathbf{v}_j$
birds	$[1, 0]$	$[1, 0]$
fly	$[0, 1]$	$[0, 2]$
south	$[1, 1]$	$[1, 1]$

Values here are 2-D as well, so $\mathbf{y}$, $\mathbf{g}$ and $\partial L / \partial \mathbf{v}_j$ are all 2-D.

Tasks

- (a) **Forward:** scores $s = \mathbf{q} \cdot \mathbf{k}_j / \sqrt{2}$, weights $\mathbf{a} = \text{softmax}(\mathbf{s})$, output $\mathbf{y} = \sum_j \mathbf{a}_j \mathbf{v}_j$.
- (b) **Step 1:** $\partial L / \partial \mathbf{V} = \mathbf{a}^T \mathbf{g}$ and $\mathbf{r} = \mathbf{V} \mathbf{g}$.
- (c) **Step 3:** the baseline $\mathbf{a}^T \mathbf{r}$, then $\partial L / \partial \mathbf{s} = \mathbf{a} \odot (\mathbf{r} - \mathbf{a}^T \mathbf{r})$. Verify that the entries sum to 0.
- (d) **Step 4:** $\partial L / \partial \mathbf{q}$ and $\partial L / \partial \mathbf{K}$ (remember the $1/\sqrt{2}$).
- (e) **Interpret:** which score does gradient descent raise the most, and why?

Hints: two of the three scores tie — what does that force on the weights? Keep 3 decimals; $\exp(0.707) \approx 2.028$, $\exp(1.414) \approx 4.113$.

Practice 2: solution

Query “fly”, $q = [1, 1]$, $g = [1, -1]$, $d_k = 2$.

(a) Forward pass

$$s = q \cdot k_j / \sqrt{2} = [1, 1, 2] / \sqrt{2} = [0.707, 0.707, 1.414]$$

$$\exp(s) = [2.028, 2.028, 4.113], \quad Z = 8.169$$

$$a = [0.248, 0.248, 0.503]$$

$$y = 0.248[1, 0] + 0.248[0, 2] + 0.503[1, 1] = [0.752, 1.000]$$

(b) Step 1 — values and r

$$\partial L / \partial V = a^T g = \text{rows } [0.248, -0.248], [0.248, -0.248], [0.503, -0.503]$$

$$r_j = g \cdot v_j: \text{ birds } 1 \cdot 1 + (-1) \cdot 0 = 1, \text{ fly } 0 - 2 = -2, \text{ south } 1 - 1 = 0$$

$$r = [1, -2, 0]$$

(c) Step 3 — score gradients

$$\text{baseline } a^T r = 0.248(1) + 0.248(-2) + 0.503(0) = -0.248$$

$$\partial L / \partial s = a \odot (r + 0.248) = [0.248(1.248), 0.248(-1.752), 0.503(0.248)]$$

$$\partial L / \partial s = [0.310, -0.435, 0.125]$$

$$\text{Check: } 0.310 - 0.435 + 0.125 = 0.000 \quad \checkmark$$

(d) Step 4 — query and keys

$$\partial L / \partial q = (1/\sqrt{2})(0.310[1, 0] - 0.435[0, 1] + 0.125[1, 1]) = (1/\sqrt{2})[0.435, -0.310]$$

$$\partial L / \partial q = [0.308, -0.219]$$

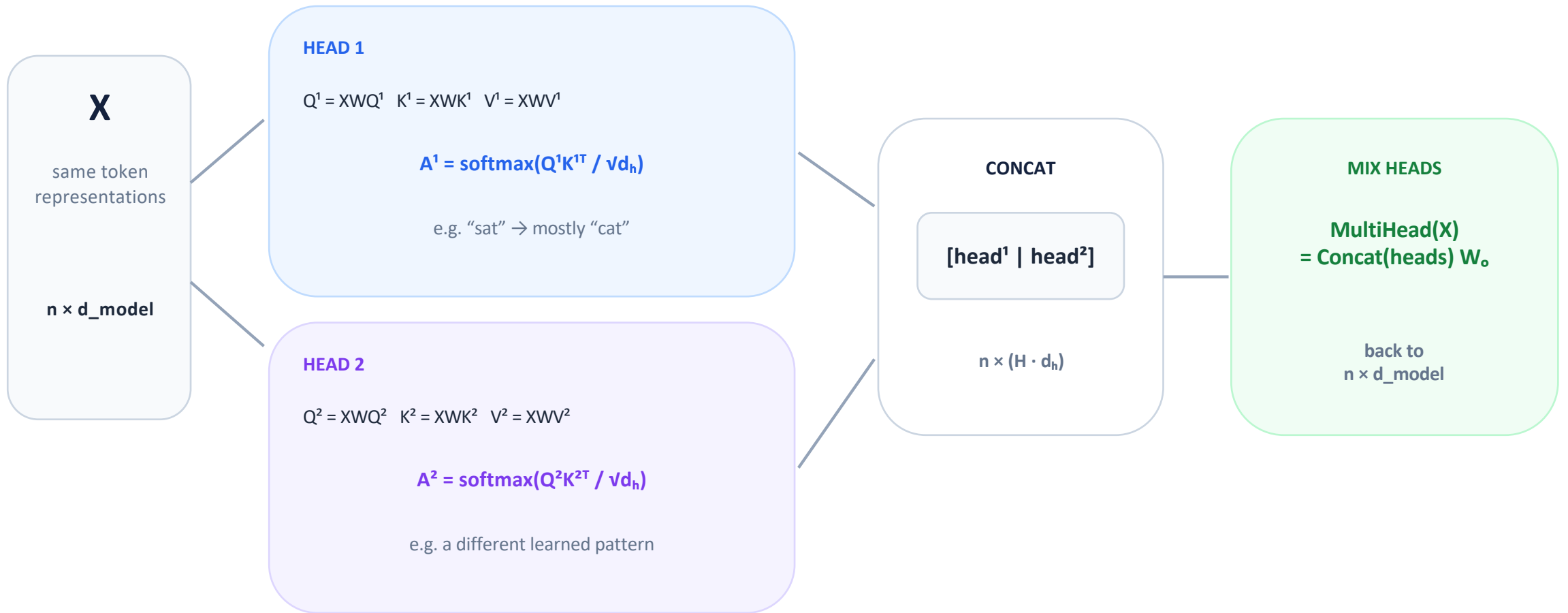
$$\partial L / \partial k_j = (\partial L / \partial s_j / \sqrt{2}) [1, 1]:$$

$$\text{birds } [0.219, 0.219], \text{ fly } [-0.308, -0.308], \text{ south } [0.088, 0.088]$$

(e) “fly” gets the most negative score gradient (−0.435): its value [0, 2] points against g , so descent raises s_{fly} and the token attends to itself more. Tied scores gave tied weights, but r breaks the tie in the gradients.

Scale the same mechanism across multiple heads

Each head gets its own learned Q/K/V projections, performs the same lookup, then the head outputs are combined.



kNNs

- Intuition: Look at my closest neighbors and try to predict
 - What already exists: a known dataset of labelled features (features, label) pairs
 - Given a new point, either classify or estimate the point value (regression)
- Close:
 - One good metric is distance (Mean square, etc.)
 - 1. Calculate distances from all my known points
 - 2. Take the **top-K neighbors (closest distance to my new point)**
 - 3. If classification, take a majority vote
 - 4. If regression, take an average

kNN Classification walkthrough

New student: 5 study hours, 4 practice tests • Use Euclidean distance • $k = 3$



Student	Study hrs	Practice tests	Pass?
A	2	1	Fail
B	3	2	Fail
C	4	4	Pass
D	6	5	Pass
E	7	3	Pass
F	8	6	Pass

KNN ALGORITHM

- 1 Choose k**
Set number of neighbors
- 2 Measure distance**
Usually Euclidean
- 3 Sort neighbors**
Closest first
- 4 Keep k nearest**
Ignore the rest
- 5 Vote**
Majority class
- 6 Return prediction**
Class label

kNN Classification walkthrough

Follow the algorithm: distance → nearest neighbors → majority vote.

STEP 2-4

Compute & rank distances

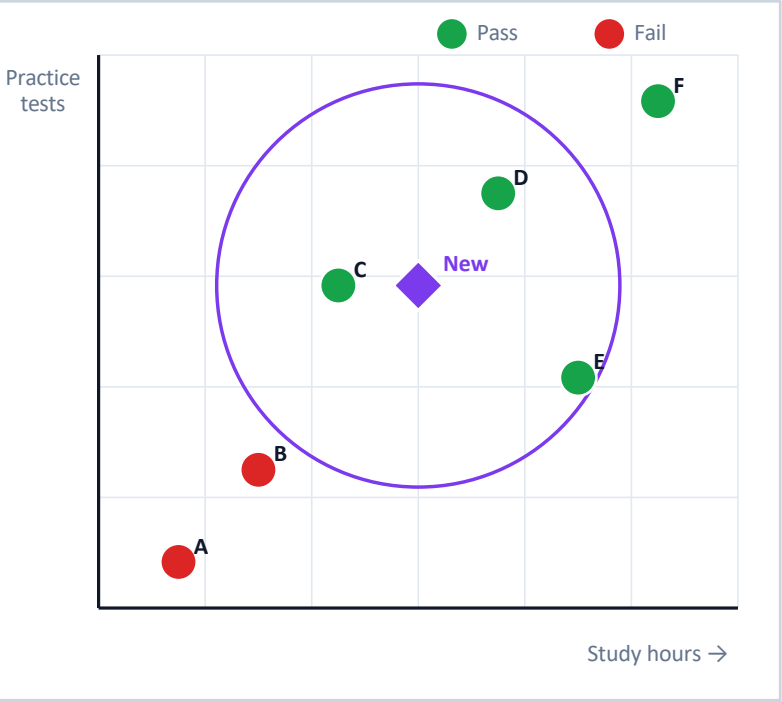
Student	Distance to (5,4)	d
C	$\sqrt{((4-5)^2+(4-4)^2)}$	1.00
D	$\sqrt{((6-5)^2+(5-4)^2)}$	1.41
E	$\sqrt{((7-5)^2+(3-4)^2)}$	2.24
B	$\sqrt{((3-5)^2+(2-4)^2)}$	2.83

STEP 5

Vote among C, D, E

C: Pass D: Pass E: Pass

3 Pass • 0 Fail



PREDICTION

PASS

KNN ALGORITHM

- 1 Choose k**
Set number of neighbors
- 2 Measure distance**
Usually Euclidean
- 3 Sort neighbors**
Closest first
- 4 Keep k nearest**
Ignore the rest
- 5 Vote**
Majority class
- 6 Return prediction**
Class label

kNN Regression walkthrough

Now the target is numeric: predict the exam score.

New student: 5 study hours, 4 practice tests • $k = 3$



Student	Study hrs	Practice tests	Score
A	2	1	48
B	3	2	55
C	4	4	72
D	6	5	82
E	7	3	78
F	8	6	90

KNN ALGORITHM

- 1 Choose k**
Set number of neighbors
- 2 Measure distance**
Usually Euclidean
- 3 Sort neighbors**
Closest first
- 4 Keep k nearest**
Ignore the rest
- 5 Average**
Mean target value
- 6 Return prediction**
Numeric value

Regression walkthrough

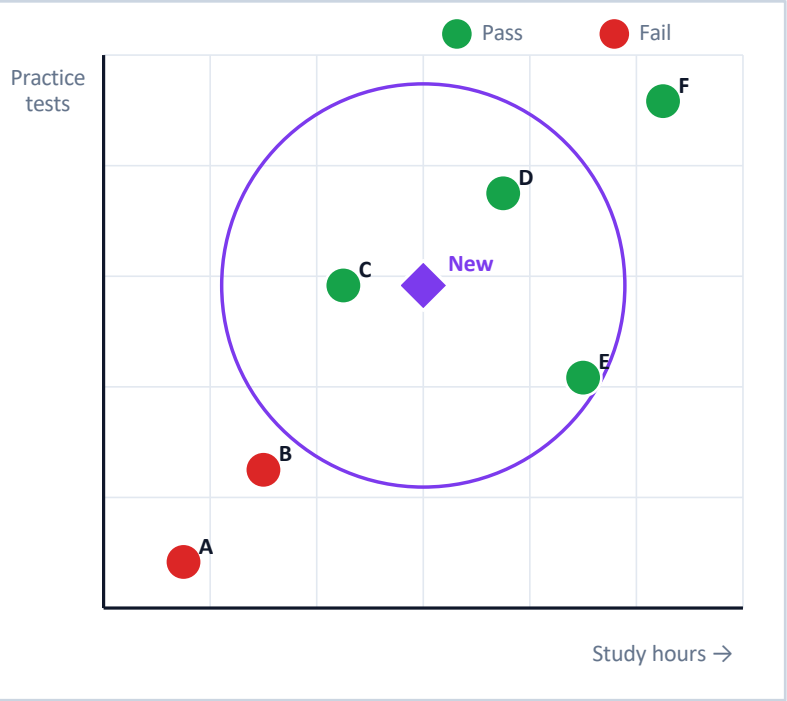
Reuse the same 3 nearest neighbors; average their numeric targets.

STEP 2-4

Nearest neighbors are unchanged

Student	Study hrs	Practice tests	Score
A	2	1	48
B	3	2	55
C	4	4	72
D	6	5	82
E	7	3	78
F	8	6	90

C, D, E are still the 3 closest points.



KNN ALGORITHM

1 Choose k
Set number of neighbors

2 Measure distance
Usually Euclidean

3 Sort neighbors
Closest first

4 Keep k nearest
Ignore the rest

5 Average
Mean target value

6 Return prediction
Numeric value

STEP 5

Average their scores

$$(72 + 82 + 78) \div 3 = 77.3$$

PREDICTION

77.3

What changes — and what stays the same?

A compact mental model for using KNN.

Same in both tasks

- 1 Choose k** controls how local/smooth the prediction is
- 2 Scale features** so one unit choice does not dominate distance
- 3 Compute distances** from the query to every training point
- 4 Select neighbors** take the k smallest distances

CLASSIFICATION

Aggregate with a vote

Output: class label

REGRESSION

Aggregate with an average

Output: numeric value

Effect of k = more local / noisier • large k = smoother / more biased